

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И КИБЕРБЕЗОПАСНОСТИ
ВЫСШАЯ ШКОЛА ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

**Отчет о прохождении научно-исследовательской (производственной)
практики**

Киричек Марии Романовны

3 курс обучения , гр. 5130203/40001

02.03.03 Математическое обеспечение и администрирование информационных
систем

Место прохождения практики: СПбПУ, ИКНК, ВШ ТИИ

г. Санкт-Петербург, ул. Обручевых, д. 1, лит. В

Сроки практики: с 15.06.2026 по 11.07.2026.

Руководитель практики от ФГАОУ ВО «СПбПУ»: Пак Вадим Геннадьевич,
к.ф.-м.н., доцент ВШТИИ.

Оценка:

Руководитель практики
от ФГАОУ ВО «СПбПУ»

В.Г. Пак

Обучающийся

М.Р. Киричек

Дата: 11.07.2026

СОДЕРЖАНИЕ

Введение	4
1 Исследование предметной области.....	6
1.1 История развития нейронного машинного перевода.....	6
1.1.1 Статистические методы	6
1.1.2 Рекуррентные нейронные сети	7
1.2 Современные методы русско-английского машинного перевода	7
1.3 Анализ подходов к построению моделей	8
1.3.1 Выбор архитектуры.....	8
1.3.2 Особенности токенизации для русско-английского перевода	9
1.4 Выводы по первой главе.....	9
2 Теоретическая часть	10
2.1 Подготовка входных данных	10
2.1.1 Токенизация (BPE и SentencePiece)	10
2.1.2 Embedding	11
2.1.3 Позиционное кодирование (Positional Encoding)	12
2.2 Архитектура Transformer: компоненты и принципы работы	13
2.3 Остаточные связи и нормализация слоёв (Add & Norm).....	15
2.4 Полносвязная нейронная сеть (Feed-Forward Network).....	17
2.5 Слои Linear и Softmax	18
2.6 Механизмы внимания (Scaled Dot-Product Attention)	20
2.6.1 Self-attention	21
2.6.2 Multi-head attention.....	21
2.6.3 Encoder-decoder attention	22
2.7 Выводы	22
3 Практическая часть	23
3.1 Подготовка корпуса OPUS-100 и предобработка данных	23
3.2 Реализация архитектуры Transformer на PyTorch	24
3.3 Инференс	24
3.4 Выводы	25
4 Апробация результатов исследования: экспериментальное исследование модели.....	25
4.1 Обучение модели	25
4.2 Оценка качества перевода: метрики BLEU, METEOR, TER, анализ ошибок	26
4.3 Выводы	28

Заключение	29
Список использованных источников.....	30
Приложение 1 Листинг программного кода.....	32

ВВЕДЕНИЕ

Задача машинного перевода является одной из важнейших задач в области естественной обработки языка (Natural Language Processing, NLP). В современном мире постоянно возникают задачи перевода текста с одного языка на другой в различных сферах деятельности от бытового общения до, например, инженерной и проектной документации.

В 2017 году исследователи из Google предложили в своей статье «Attention is all you need» архитектуру модели искусственной нейронной сети (ИНС) "Transformer". Отличительной особенностью их работы стала разработка первой модели, в основе которой они ушли от традиционных рекуррентных слоёв, используемых в encoder-decoder архитектурах, к механизмам внимания [4]. Данная архитектура лежит в основе современных систем искусственного интеллекта (СИИ), например, в системах машинного перевода текста и диалоговых агентах (например, Claude Sonnet). Часто в задачах машинного перевода с одного языка на другой СИИ могут быть представлены диалоговыми ассистентами или ИИ-агентами [найти вставить про это источник].

Актуальность исследования заключается в необходимости постоянного развития и совершенствования методов машинного перевода. В частности, к такому методу и относится архитектура Transformer, которая будет реализована в данной работе для машинного русско-английского перевода текста. Понимание механизмов работы этой архитектуры остаётся важным для формирования фундаментальных навыков проектирования нейросетевых архитектур.

Объект исследования — процесс машинного перевода текстов с русского языка на английский.

Предмет исследования — архитектура Transformer, её компоненты, методы обучения и оценки качества перевода.

Цель исследования — реализовать и экспериментально исследовать архитектуру Transformer для задачи русско-английского машинного перевода.

В соответствии с целью исследования, логически определяются следующие **задачи работы**:

- А. Изучить архитектуру Transformer.
- В. Выполнить подготовку русско-английского датасета OPUS-100: провести токенизацию, нормализацию текста и фильтрацию выбросов, разбиение на обучающую, валидационную и тестовую выборки.

- C. Реализовать архитектуру Transformer с нуля на фреймворке PyTorch.
- D. Провести обучение модели, экспериментальное исследование влияния гиперпараметров на качество перевода, измеряемое метриками BLEU, METEOR и TER.
- E. Выполнить тестирование обученной модели, провести анализ ошибок перевода.
- F. Сформулировать выводы о применимости полученных знаний и навыков для разработки диалогового ассистента для перевода текстов с русского языка на английский.

1 ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

Глава посвящена исследованию теоретических основ и существующих подходов в области машинного русско-английского перевода. Приводится история развития нейронного машинного перевода (параграф 1.1), рассматриваются современные системы перевода (параграф 1.2), а также выполняется сравнительный анализ архитектур и методов токенизации (параграф 1.3).

1.1 История развития нейронного машинного перевода

1.1.1 Статистические методы

Первые системы машинного перевода (Machine Translation, MT), появившиеся в середине XX века, использовали лингвистические правила (Rule-Based MT, RBMT) [10]. Они разрабатывались под узкие задачи — например, для перевода военных отчетов или научных публикаций.

Однако со временем сферы применения технологии расширились. На рубеже 1980–1990-х годов возникла необходимость массово переводить бизнес-документы, международные новости и объёмную техническую документацию. Правилые системы (RBMT) не справлялись с такими масштабами, так как составление словарей вручную требовало слишком много времени. Это привело к переходу на статистический машинный перевод (Statistical MT, SMT), который работал на основе автоматического анализа больших параллельных корпусов данных [11; 17].

Математической основой стандартного SMT служит теорема Байеса. В поиске наилучшего перевода $\hat{e}$ система максимизирует произведение двух вероятностей: $P(f|e)$ (правдоподобие перевода) и $P(e)$ (языковая естественность):

$$\hat{e} = \arg \max_e (P(f|e) \cdot P(e)) \quad (1.1)$$

За первый сомножитель в этой формуле отвечает модель выравнивания (alignment model), которая сопоставляет слова исходника и перевода, проверяя точность передачи смысла. За второй — языковая модель, которая на основе анализа n -грамм оценивает грамматическую корректность и плавность итогового предложения.

Развитием этого подхода стал фразовый перевод (Phrase-Based SMT). В нем вместо отдельных слов система сопоставляла целые фразы, объединяя параметры через логарифмически линейную (log-linear) комбинацию весов.

Главным недостатком SMT было качество работы с морфологически богатыми языками, включая русский. Статистические модели не учитывали широкий контекст и не умели правильно согласовывать падежи, роды и окончания, из-за чего в готовом тексте часто возникали грубые грамматические ошибки.

1.1.2 Рекуррентные нейронные сети

Развитие направления нейронного машинного перевода (Neural MT, NMT) в 2014–2016 годах обусловлено прогрессом в области глубокого обучения и внедрением архитектуры Encoder-Decoder на базе рекуррентных нейронных сетей (RNN) [12—14; 17].

В рамках данной концепции кодировщик (Encoder) на основе блоков LSTM (Long Short-Term Memory — долгая краткосрочная память) или GRU (Gated Recurrent Unit — управляемый рекуррентный блок) сжимает входную последовательность в вектор контекста фиксированной размерности, а декодировщик (Decoder) пошагово генерирует целевой текст [8; 15; 16]. Ключевым ограничением классических RNN-NMT систем являлась деградация качества перевода при увеличении длины предложений [14; 18]. Эта проблема была решена интеграцией механизма внимания (Attention Mechanism), позволившего декодировщику динамически фокусироваться на информативных частях исходной последовательности [2; 4; 18]. Успешное внедрение систем нового типа в промышленную эксплуатацию (в частности, Google GNMT в 2016 году) окончательно доказало превосходство нейросетевого подхода над статистическим [10—12].

1.2 Современные методы русско-английского машинного перевода

Текущим технологическим стандартом в нейронном машинном переводе (NMT) является архитектура Transformer, основанная на механизмах самовнимания (Self-Attention) [2]. Применительно к русско-английской языковой паре (Ru-En) современные методы можно разделить на три основные группы, каждая из которых решает свои практические задачи [13]:

- A. **Мультиязычные модели (Multilingual NMT):** Универсальные сети (такие как mBART, M2M-100 или NLLB от Meta), которые обучаются на сотнях языков одновременно и используют перенос знаний (transfer learning). Их главное преимущество — способность переводить между редкими языковыми парами. Однако на популярных направлениях вроде Ru-En они часто уступают специализированным решениям в точности формулировок.
- B. **Специализированные открытые Ru-En модели:** Системы, обученные целенаправленно на русско-английских корпусах (например, модели MarianMT на платформе Hugging Face или фреймворки OpenNMT). Благодаря открытому исходному коду такие решения востребованы в академических исследованиях и коммерческих проектах, где необходимо дообучить модель под конкретную предметную область — например, под медицинские тексты или юридическую документацию.
- C. **Коммерческие экосистемы:** Интегрированные облачные платформы от крупных технологических компаний, ориентированные на массового пользователя и бизнес. Так, Яндекс.Переводчик использует гибридные подходы, адаптированные под сложную морфологию русского языка; Google Translate нацелен на масштабную мультиязычность [8]; а DeepL фокусируется на точной передаче контекста в корпоративном и художественном переводе [15].

1.3 Анализ подходов к построению моделей

1.3.1 Выбор архитектуры

Выбор нейросетевой архитектуры определяет качество перевода и скорость работы модели. Сравнение классических рекуррентных сетей (RNN) и современного Трансформера [18] приведено в таблице 1.1.

Главное преимущество Трансформера — механизм самовнимания (Self-Attention) [2]. Он позволяет модели одновременно обрабатывать все слова в предложении и находить связи между ними, независимо от длины текста.

Таблица 1.1

Сравнение архитектур RNN и Transformer

Критерий	Рекуррентные сети (RNN)	Архитектура Transformer
Обработка текста	По очереди (слово за словом)	Одновременно (весь текст сразу)
Скорость обучения	Медленная (нельзя распараллелить)	Высокая (за счет параллельных вычислений)
Длинный контекст	Теряет смысл в длинных фразах	Хорошо помнит весь текст
Качество перевода	Часто ошибается в сложных фразах	Высокая точность и связность текста
Требования к памяти	Низкие требования при работе	Высокие требования к видео-памяти

1.3.2 Особенности токенизации для русско-английского перевода

Перед подачей в модель текст необходимо разбить на части — токены [3]. В паре русский-английский это сложно сделать обычным делением по словам. В английском языке слова почти не меняются, а в русском — огромное количество окончаний, суффиксов и приставок. Из-за этого словарь модели быстро разрастается, и в него постоянно попадают незнакомые слова.

Чтобы решить эту проблему, используют подсловную токенизацию (subwords) [7]. Алгоритмы (например, популярный BPE) делят редкие и сложные слова на кусочки (корни и суффиксы).

Например, редкое для модели слово «подводный» алгоритм разобьет на части: ["под", "##вод", "##ный"]. В итоге модель понимает смысл слова по его частям, даже если никогда не видела его целиком. Это критически важно для корректной обработки русского языка.

1.4 Выводы по первой главе

- А. Эволюция систем машинного перевода от RBMT и SMT к нейросетевым архитектурам (NMT) существенно повысила точность и естественность генерации текста [10; 11].
- В. Архитектура Transformer является стандартом в задачах NMT, превосходя рекуррентные сети в скорости обучения и работе с длинным контекстом за счет механизма Self-Attention [2; 18].

- С. Сложная морфология русского языка обуславливает необходимость применения методов подсловной токенизации (в частности, алгоритма BPE) для решения проблемы незнакомых слов [3; 7].
- Д. Результаты анализа обосновывают самостоятельную программную реализацию модели Transformer на фреймворке PyTorch и предварительную фильтрацию выбранного параллельного корпуса данных.

2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В главе рассматриваются теоретические основы архитектуры Transformer: подготовка входных данных (токенизация, эмбединги и позиционное кодирование), общая структура модели, механизмы внимания, нормализация слоёв, полносвязные сети. В параграфе 2.7 приведены выводы по главе.

2.1 Подготовка входных данных

Перед тем как текст может быть обработан архитектурой Transformer, он проходит несколько этапов подготовки. Поскольку нейронные сети способны работать только с числовыми данными, исходный текст необходимо преобразовать в последовательность векторов. Данный процесс включает три основных этапа: токенизацию, преобразование токенов в эмбединги и добавление информации о положении каждого токена в последовательности с помощью позиционного кодирования.

2.1.1 Токенизация (*BPE и SentencePiece*)

Токенизация представляет собой процесс разбиения текста на отдельные элементы — токены, которые впоследствии используются моделью в качестве входных данных. В простейшем случае токенами могут являться отдельные слова, однако такой подход приводит к возникновению проблемы большого словаря и невозможности корректной обработки неизвестных слов.

Для решения данной проблемы в современных моделях машинного перевода широко применяются алгоритмы субсловной токенизации [9]. Вместо хранения каждого слова целиком модель разбивает слова на более мелкие фрагменты (суб-

слова, подслова), из которых впоследствии могут быть составлены как известные, так и ранее не встречавшиеся слова.

Одним из наиболее распространённых методов является алгоритм Byte Pair Encoding (BPE). Первоначально каждое слово представляется последовательностью отдельных символов. Затем наиболее часто встречающиеся соседние пары символов постепенно объединяются в новые токены. После большого числа итераций формируется словарь наиболее часто встречающихся подслов.

Другим популярным методом является SentencePiece. В отличие от BPE, данный алгоритм работает непосредственно с необработанным текстом и не требует предварительного деления предложений на слова. Благодаря этому SentencePiece получил широкое распространение в современных языковых моделях.

2.1.2 Embedding

После выполнения токенизации каждый полученный токен заменяется числовым идентификатором, который затем преобразуется в вектор фиксированной размерности с помощью слоя Embedding. Данный слой представляет собой обучаемую матрицу параметров, в которой каждому токenu словаря соответствует собственный вектор признаков.

Пусть размер словаря равен $|V|$, а размерность пространства признаков — d_{model} . Тогда матрица эмбедингов имеет размерность:

$$W_E \in \mathbb{R}^{|V| \times d_{model}}. \quad (2.1)$$

Для каждого токена с индексом x_i выбирается соответствующая строка матрицы:

$$e_i = W_E[x_i]. \quad (2.2)$$

В результате вся последовательность токенов преобразуется в последовательность векторов размерности d_{model} .

Следует отметить, что значения эмбедингов не задаются заранее. Они обучаются совместно со всеми остальными параметрами модели методом обратного распространения ошибки. Благодаря этому слова, имеющие схожее значение или употребляющиеся в похожем контексте, получают близкие векторные представления.

В оригинальной архитектуре Transformer размерность эмбеддингов совпадает с размерностью внутренних представлений модели и составляет:

$$d_{model} = 512. \quad (2.3)$$

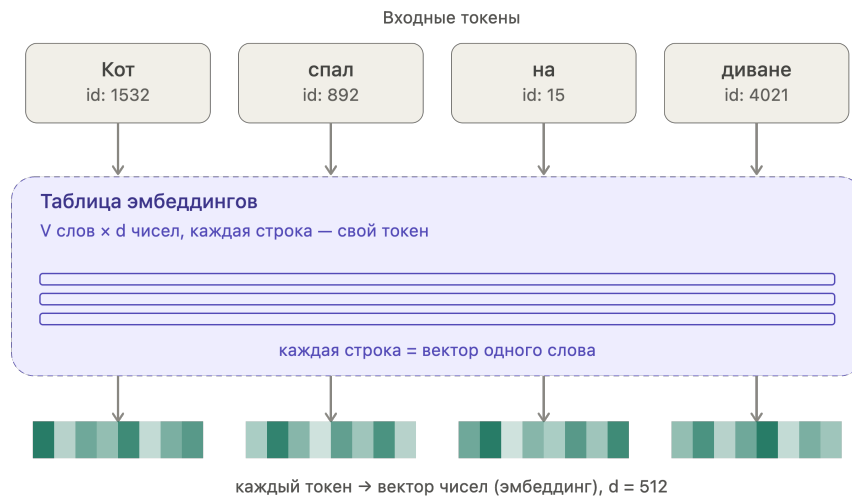


Рис.2.1. Преобразование токенов в эмбеддинги

2.1.3 Позиционное кодирование (*Positional Encoding*)

В отличие от рекуррентных нейронных сетей, архитектура Transformer одновременно обрабатывает всю последовательность токенов и не обладает встроенной информацией об их порядке. Поэтому к эмбеддингам дополнительно добавляется позиционное кодирование, позволяющее модели учитывать расположение слов в предложении.

В оригинальной работе используется синусоидальное позиционное кодирование, вычисляемое по формулам:

$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{\frac{2i}{d_{model}}}} \right). \quad (2.4)$$

$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{\frac{2i}{d_{model}}}} \right). \quad (2.5)$$

где pos — позиция токена в последовательности, а i — индекс компоненты вектора.

Использование синусоидальных функций позволяет получать уникальные позиционные представления для последовательностей произвольной длины. Кроме

того, благодаря различным частотам синуса и косинуса модель способна учитывать как близкие, так и удалённые взаимные положения слов.

После вычисления позиционный вектор складывается с соответствующим эмбеддингом:

$$X = E + PE. \quad (2.6)$$

Именно полученные после этого представления поступают на вход первого слоя кодировщика Transformer.

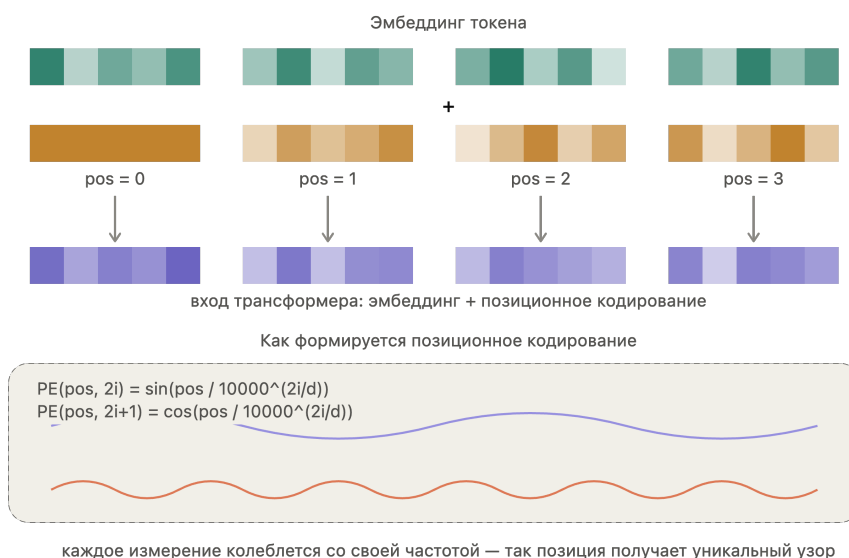


Рис.2.2. Добавление позиционного кодирования к эмбеддингам

Благодаря этому модель получает информацию не только о содержании токенов, но и об их относительном положении в последовательности.

2.2 Архитектура Transformer: компоненты и принципы работы

Для начала рассмотрим архитектуру Transformer на абстрактном уровне, не углубляясь во внутреннее устройство её компонентов. Для этого посмотрим на модель как на чёрный ящик. В приложении для перевода с одного языка на другой на ввод будет подаваться предложение на первом языке, а на выводе будет оно же, переведённое на второй язык.

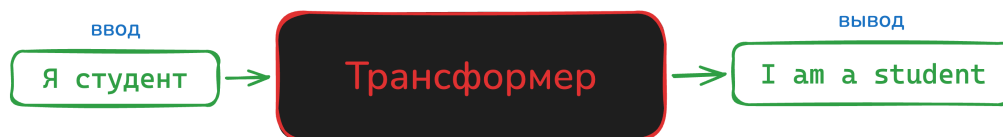


Рис.2.3. Архитектура Transformer [2]

При более детальном рассмотрении архитектуры можно выделить компонент кодирования, компонент декодирования и связи между ними.

Компонент кодирования представляет собой стек кодировщиков (в статье "The attention is all you need"[4] их 6). Компонент декодирования представляет собой стек декодеров. Между кодерами и декодерами есть связи.

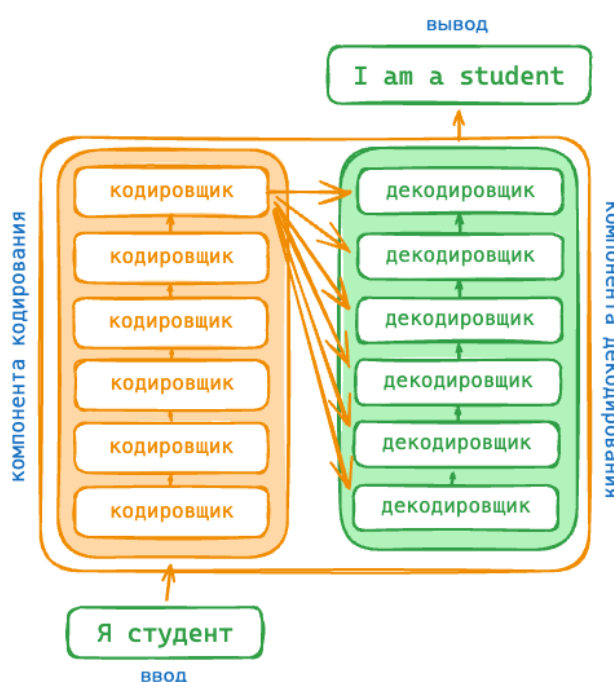


Рис.2.4. Более подробная архитектура Transformer [2]

Рассмотрим подробнее структуру кодировщика. Все кодировщики идентичны по структуре (но у них нет общих весов).

Каждый кодировщик состоит из нескольких функциональных блоков, каждый из которых выполняет определённую задачу при обработке входной последовательности. Входные данные кодировщика сначала проходят через self-attention (самовнимание) — слой, который помогает кодировщику просматривать другие слова в предложении, которое подаётся на ввод, при кодировании *какого-то определённого слова*. Выходные данные слоя self-attention передаются в нейронную сеть feed forward (это простейший многослойный перцептрон, в нём информация движется от входа к выходу, прямо без циклов и обратных связей). Одна и та же

сеть feed forward независимо применяется к каждой позиции. Строение данных двух слоёв будет подробно рассматриваться в следующих параграфах.

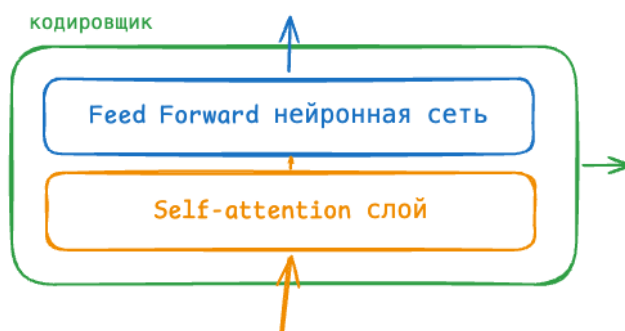


Рис.2.5. Кодировщик [2]

Теперь рассмотрим декодер. В декодере есть оба этих слоя, но между ними находится слой внимания, который помогает декодеру сосредоточиться на соответствующих частях входного предложения.

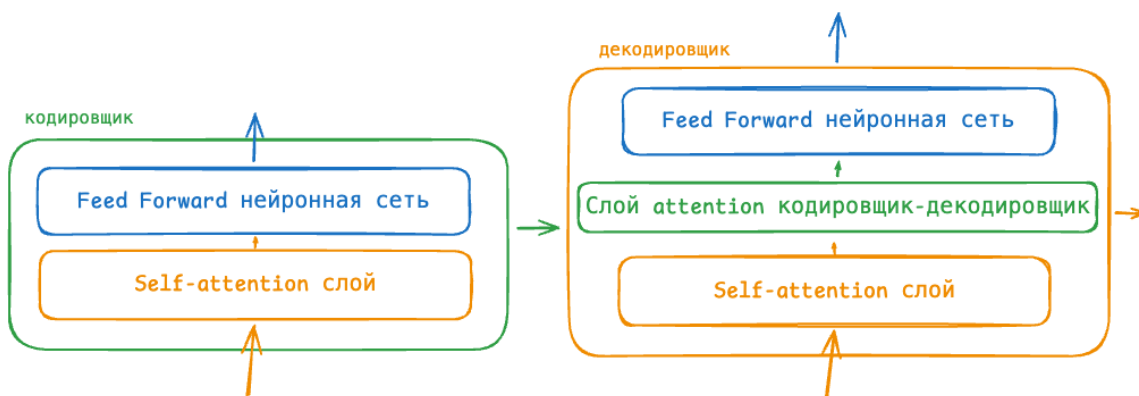


Рис.2.6. Кодировщик-декодировщик [2]

Для корректной работы каждого блока архитектуры используются дополнительные механизмы, обеспечивающие устойчивость и эффективность обучения модели. Одним из таких механизмов является операция Add & Norm.

2.3 Остаточные связи и нормализация слоёв (Add & Norm)

В оригинальной архитектуре Transformer после каждого подслоя применяется операция *Add & Norm*, которая описывается формулой:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2.7)$$

где x — вход в подслой, а $\text{Sublayer}(x)$ — выход подслоя (механизма внимания или полносвязной сети).

Остаточная связь (сложение $x + \text{Sublayer}(x)$) позволяет градиентам распространяться через сеть напрямую, решая проблему затухающих градиентов. Слой нормализации LayerNorm стабилизирует обучение, приводя активации к нулевому среднему и единичной дисперсии [4].

Архитектура кодировщика с данными слоями представлена на рисунке 2.7.

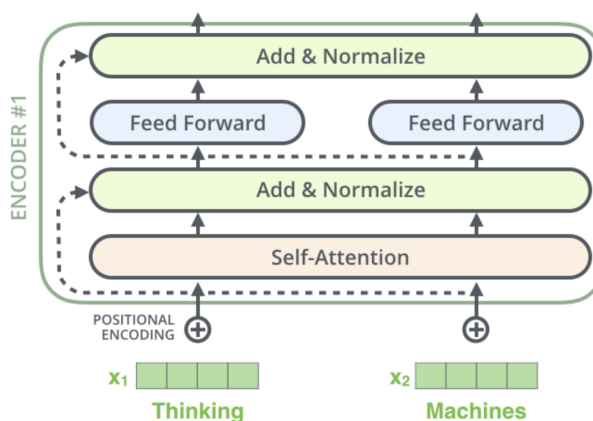


Рис.2.7. full encoder

Архитектура кодировщика с данными слоями, но уже более подробно, представлена на рисунке 2.8.

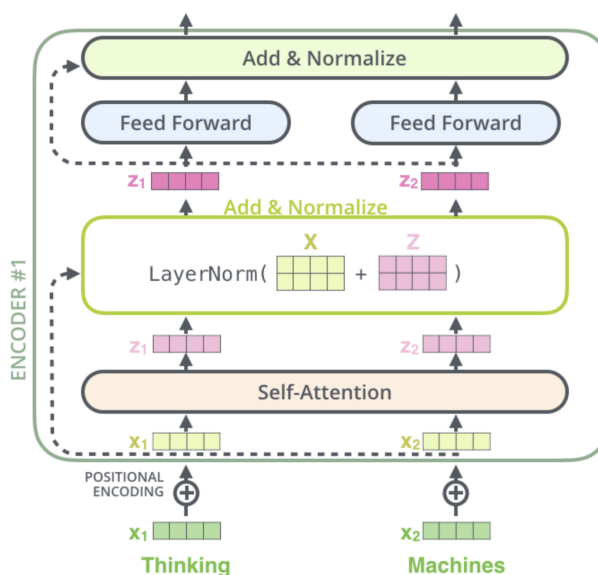


Рис.2.8. detailed encoder

Полная архитектура модели с кодировщиком и декодировщиком представлена на рисунке 2.9.

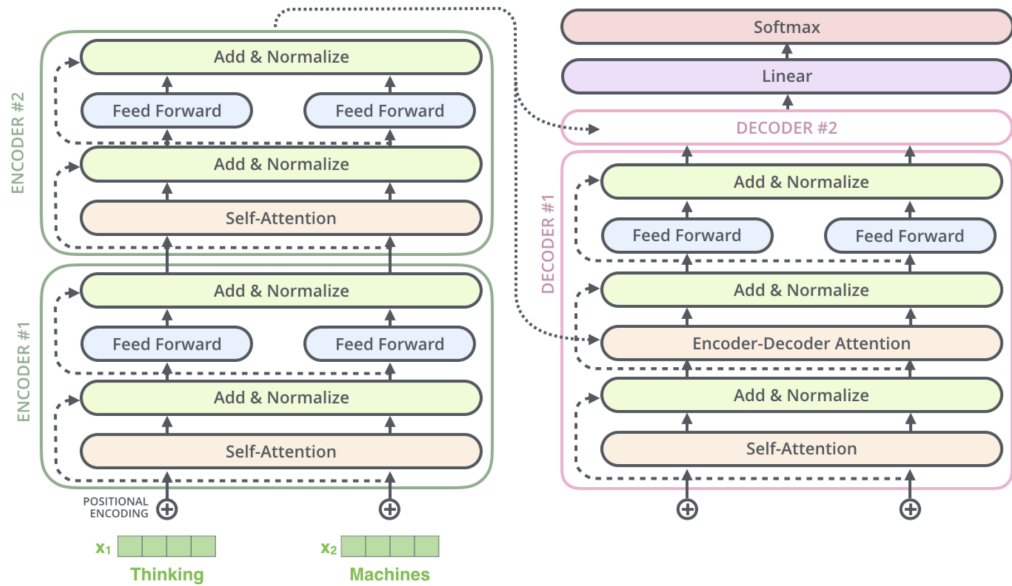


Рис.2.9. Подробная архитектура

Здесь два кодировщика и декодировщика, а также присутствует слой Linear и Softmax — они нужны для преобразования выходного вектора декодера в распределение вероятностей по словарю целевого языка. Линейный слой проектирует вектор размерности d_{model} в вектор размерности, равной размеру целевого словаря. Softmax преобразует полученные значения в вероятности, сумма которых равна единице. Слово с максимальной вероятностью выбирается в качестве очередного слова перевода на каждом шаге генерации [4].

Для понимания работы каждого блока необходимо рассмотреть дополнительные компоненты, используемые после каждого слоя.

2.4 Полносвязная нейронная сеть (Feed-Forward Network)

После каждого слоя внимания в архитектуре Transformer применяется полносвязная сеть, которая состоит из двух линейных преобразований с функцией активации ReLU между ними. Этот процесс можно представить в два этапа:

Сначала первый линейный слой преобразует входной вектор x размерности d_{model} в пространство размерности d_{ff} :

$$h = \text{ReLU}(xW_1 + b_1), \quad h \in \mathbb{R}^{d_{\text{ff}}} \quad (2.8)$$

Затем второй линейный слой возвращает вектор к исходной размерности:

$$\text{FFN}(x) = hW_2 + b_2, \quad \text{FFN}(x) \in \mathbb{R}^{d_{\text{model}}} \quad (2.9)$$

где $\text{ReLU}(z) = \max(0, z)$, $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ — матрицы весов, а $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$ — векторы смещений.

В нашем случае $d_{\text{model}} = 512$ и $d_{\text{ff}} = 2048$ [4]. После применения функции активации ReLU второй линейный слой возвращает вектор к исходной размерности $d_{\text{model}} = 512$. Такая архитектура позволяет модели выявлять более сложные нелинейные взаимосвязи, расширяя представление признаков в более высокоразмерное пространство, а затем сжимая его обратно. Важно отметить, что полносвязная сеть применяется независимо к каждой позиции в последовательности.

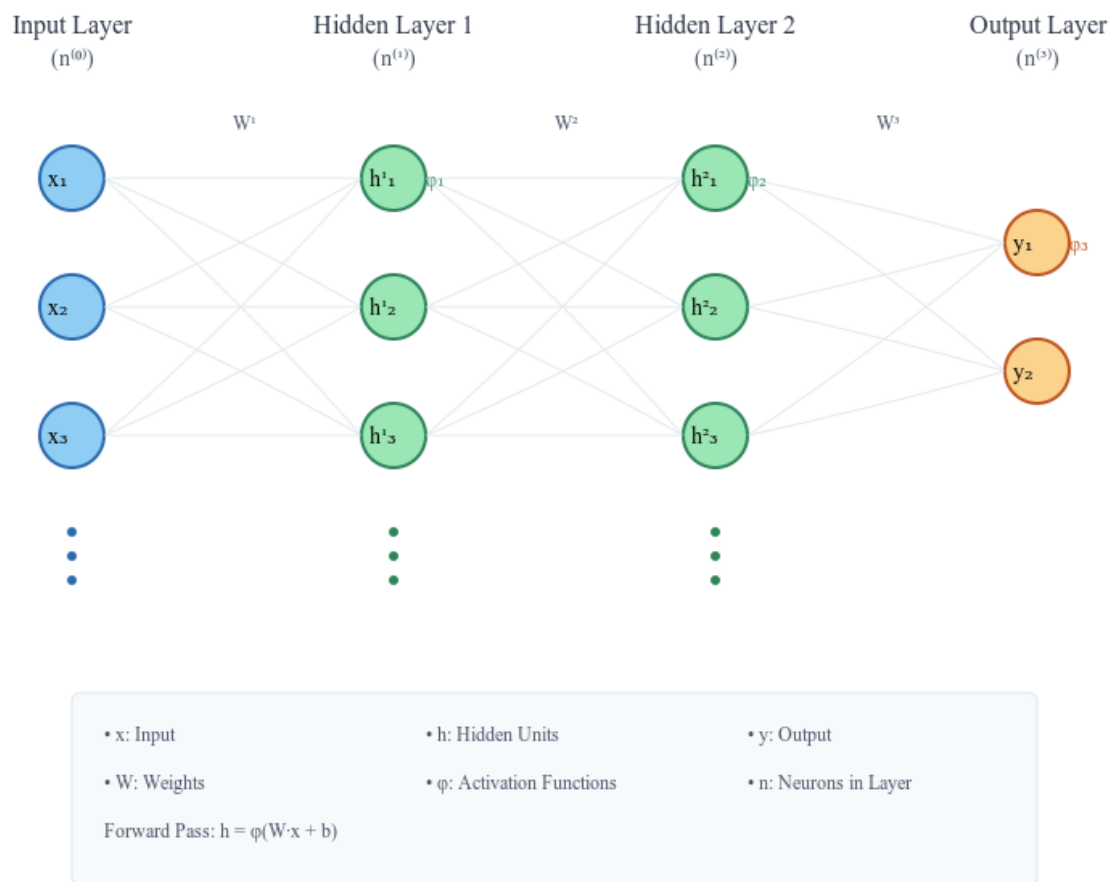


Рис.2.10. Полносвязная нейронная сеть (Feed-Forward Neural Network)

2.5 Слои Linear и Softmax

После прохождения последовательности через стек декодеров на выходе формируется вектор признаков размерности (d_{model}). Однако данный вектор ещё не соответствует конкретному слову целевого языка и требует дополнительного преобразования.

Для этой цели используется линейный слой (Linear), который выполняет проекцию выходного вектора декодера в пространство размерности, равной объёму словаря целевого языка. В результате формируется вектор логитов, в котором каждому элементу соответствует одно слово словаря. Например, если словарь содержит 10 000 уникальных слов, то линейный слой сформирует вектор из 10 000 значений.

Полученные логиты сами по себе не являются вероятностями, поэтому далее применяется функция Softmax. Она преобразует значения логитов в распределение вероятностей, при котором все значения находятся в диапазоне от 0 до 1, а их сумма равна единице. Каждая вероятность отражает степень уверенности модели в выборе соответствующего слова.

На этапе генерации перевода в качестве следующего слова обычно выбирается токен, имеющий наибольшую вероятность. После этого выбранный токен передаётся на следующий шаг декодирования для формирования последующих слов предложения.

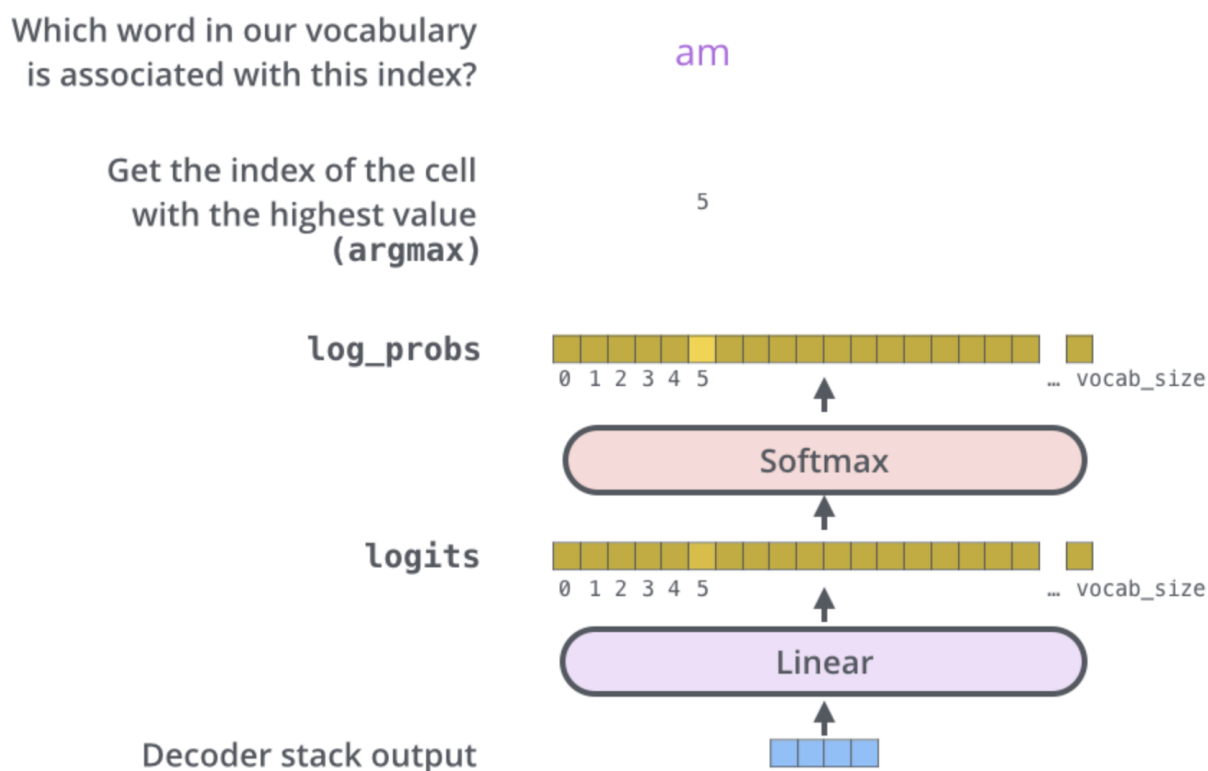


Рис.2.11. Слои Linear и Softmax

Однако ключевым элементом архитектуры Transformer остаётся механизм внимания, определяющий способ формирования контекстных представлений слов.

2.6 Механизмы внимания (Scaled Dot-Product Attention)

Одной из ключевых особенностей архитектуры Transformer является механизм внимания (Attention), благодаря которому модель может учитывать взаимосвязи между словами независимо от их положения в последовательности. В отличие от рекуррентных нейронных сетей, где информация передаётся последовательно от одного слова к другому, механизм внимания позволяет одновременно анализировать всю входную последовательность. Это существенно повышает эффективность обработки длинных предложений и облегчает выявление как синтаксических, так и семантических зависимостей между словами.

Основой механизма внимания являются три вектора: запрос (Query, Q), ключ (Key, K) и значение (Value, V). Они вычисляются с помощью линейных преобразований входных представлений:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2.10)$$

где X — матрица входных эмбеддингов, а W_Q , W_K и W_V — обучаемые матрицы весов.

Далее вычисляется матрица оценок внимания как скалярное произведение матриц запросов и ключей:

$$QK^T, \quad (2.11)$$

Полученные значения масштабируются с целью предотвращения слишком больших значений при увеличении размерности пространства признаков:

$$\frac{QK^T}{\sqrt{d_k}}, \quad (2.12)$$

где d_k — размерность векторов ключей.

После масштабирования применяется функция Softmax, преобразующая оценки внимания в вероятностное распределение:

$$\text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right), \quad (2.13)$$

Итоговое представление вычисляется по формуле:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (2.14)$$

Полученные коэффициенты внимания определяют степень влияния каждого слова последовательности на формирование нового представления текущего слова.

В архитектуре Transformer используются три разновидности механизма внимания: self-attention, multi-head attention и encoder-decoder attention. Несмотря на использование общей формулы вычисления внимания, каждая из них выполняет собственную функцию.

2.6.1 Self-attention

Механизм self-attention применяется как в кодировщике, так и в декодере и позволяет каждому слову учитывать информацию обо всех остальных словах той же последовательности. Благодаря этому модель способна выявлять зависимости между словами независимо от расстояния между ними.

Особенностью данного механизма является то, что запросы, ключи и значения формируются из одной и той же последовательности:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (2.15)$$

В результате каждое слово получает новое представление, сформированное с учётом контекста всего предложения. Такой подход позволяет модели учитывать как близкие, так и удалённые зависимости между словами.

В кодировщике self-attention вычисляется без ограничений, тогда как в декодере используется маскирование (masked self-attention), запрещающее учитывать будущие токены последовательности. Это позволяет модели генерировать перевод последовательно, не используя информацию о ещё не сгенерированных словах.

2.6.2 Multi-head attention

Использование только одного механизма внимания ограничивает способность модели выявлять различные типы взаимосвязей между словами. Поэтому в архитектуре Transformer применяется механизм Multi-Head Attention, при котором вычисляются несколько механизмов внимания параллельно.

Для каждой головы внимания вычисляется собственный результат:

$$head_i = \text{Attention}(Q_i, K_i, V_i), \quad (2.16)$$

где $i = 1, \dots, h$, а h — количество голов внимания.

После вычисления всех голов их результаты объединяются операцией конкатенации и проходят через дополнительное линейное преобразование:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad (2.17)$$

где W_O — обучаемая матрица весов.

Использование нескольких голов внимания позволяет модели одновременно анализировать различные аспекты входной последовательности. Одни головы могут выявлять синтаксические зависимости между словами, другие — семантические связи, что повышает качество формируемых представлений.

2.6.3 Encoder-decoder attention

Помимо механизма self-attention, в декодере используется механизм encoder-decoder attention. Его основная задача заключается в передаче информации от кодировщика к декодеру при генерации перевода.

В отличие от self-attention, запросы формируются выходом декодера, тогда как ключи и значения поступают из выхода кодировщика:

$$Q = \text{Decoder}, \quad K = \text{Encoder}, \quad V = \text{Encoder}. \quad (2.18)$$

После формирования запросов, ключей и значений вычисление внимания выполняется по формуле (2.14).

Благодаря этому механизму декодер определяет, на какие части исходного предложения следует обратить наибольшее внимание при генерации очередного слова перевода. Это обеспечивает согласованность между входной и выходной последовательностями и позволяет формировать более точный перевод.

2.7 Выводы

В данной главе были рассмотрены теоретические основы архитектуры Transformer, используемой для решения задач машинного перевода. Изучены этапы подготовки входных данных, включая токенизацию, преобразование токенов в эмбединги и позиционное кодирование. Рассмотрены основные компоненты архитектуры Transformer: механизм внимания, многоголовое внимание, остаточные связи, нормализация слоёв, полносвязная нейронная сеть и выходные слои Linear и Softmax. Полученные теоретические сведения являются основой для

последующей практической реализации модели машинного перевода и проведения экспериментального исследования.

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

Глава посвящена практической реализации модели машинного перевода. Рассматриваются подготовка датасета OPUS-100 и предобработка данных (параграф 3.1), а также реализация архитектуры Transformer на PyTorch (параграф 3.2). В параграфе 3.4 приведены выводы по главе.

3.1 Подготовка корпуса OPUS-100 и предобработка данных

В качестве обучающего корпуса был выбран датасет OPUS-100 (пара языков "русский-английский"), загруженный через библиотеку `datasets`. Он содержит 1 миллион тренировочных, 2000 валидационных и 2000 тестовых пар предложений.

Анализ данных выявил наличие артефактов — китайских иероглифов, символов индийского алфавита и других посторонних знаков. Для фильтрации таких примеров была реализована функция `is_clean`, которая с помощью регулярных выражений пропускает только текст, состоящий из латиницы, кириллицы, цифр и базовых знаков препинания. Это убирает шум из обучающей выборки.

Токенизация выполнена с использованием алгоритма Byte Pair Encoding (BPE) из библиотеки `tokenizers`. Словарь содержит 32 000 токенов, включая специальные токены: `[PAD]` для выравнивания последовательностей, `[UNK]` для неизвестных слов, `[BOS]` и `[EOS]` для маркеров начала и конца предложения. Токенизатор обучался на отфильтрованных данных.

На этапе подготовки числовых данных предложения обрезаются до 200 токенов. Во входную последовательность добавляется токен `[EOS]`, в целевую — `[BOS]` в начале и `[EOS]` в конце. Для выравнивания длин в пределах батча реализован класс `DataCollator`, который динамически дополняет последовательности токенами `[PAD]` до максимальной длины в батче.

Для корректной работы механизма внимания формируются маски: `src_padding_mask` и `tgt_padding_mask` указывают на позиции с `[PAD]`-токенами, а каузальная маска (`generate_square_subsequent_mask`) блокирует доступ

декодера к будущим токенам. Загрузка данных выполняется через DataLoader с размером батча 4, с перемешиванием для тренировочной выборки.

3.2 Реализация архитектуры Transformer на PyTorch

Архитектура Transformer реализована с нуля на PyTorch в соответствии с оригинальной спецификацией из статьи "Attention Is All You Need".

Позиционное кодирование добавлено через класс `PositionalEncoding`, использующий синусоидальные функции для внесения информации о порядке токенов в последовательности.

Механизм многоголового внимания (`MultiHeadAttention`) реализован с разделением на $n_{\text{head}} = 8$ голов, каждая размерности $d_k = d_{\text{model}}/n_{\text{head}}$. Для каждой головы вычисляется scaled dot-product attention, результаты конкатенируются и проецируются через линейный слой. Это позволяет модели одновременно учитывать различные аспекты взаимосвязей между токенами.

Слой энкодера (`EncoderLayer`) включает многоголовое self-attention и полносвязную сеть (`FeedForward`) с размерностью скрытого слоя 2048, каждый компонент дополнен остаточной связью и `LayerNorm`. Слой декодера (`DecoderLayer`) содержит masked self-attention, cross-attention к выходу энкодера и полносвязную сеть с аналогичной структурой.

Итоговая модель (`SimpleTransformer`) объединяет слой эмбеддингов, позиционное кодирование, стек из 6 слоёв энкодера и 6 слоёв декодера, а также выходной линейный слой для проекции в размер словаря. Размерность модели d_{model} установлена в 512.

Обучение ведётся с функцией потерь `CrossEntropyLoss` с игнорированием [PAD]-токенов. Оптимизатор — Adam. Для работы с ограниченной видеопамятью применено накопление градиентов за 8 шагов, что эквивалентно эффективному батчу 32. Чекпоинты сохраняются каждые 2000 батчей; лучшая модель определяется по минимальному значению валидационного loss.

3.3 Инференс

Входное предложение токенизируется, дополняется токеном [EOS] и подается в энкодер, который формирует закодированное представление исходного текста. Декодер инициализируется токеном [BOS] и на каждом шаге вычисля-

ет следующий токен на основе уже сгенерированной части и выходных данных энкодера. На каждом шаге выбирается токен с максимальной вероятностью, что обеспечивает детерминированность результата. Генерация останавливается при получении токена [EOS] или достижении максимальной длины последовательности (100 токенов). После завершения генерации из последовательности удаляется начальный токен [BOS], и оставшиеся токены декодируются в текст с помощью токенизатора. Генерация перевода выполняется авторегрессивно.

3.4 Выводы

В рамках практической части выполнена реализация модели машинного перевода на основе Transformer. Подготовлен датасет OPUS-100, обучен BPE-токенизатор, реализованы все компоненты архитектуры, настроено обучение с градиентным накоплением и валидацией, реализован авторегрессивный инференс. Модель работоспособна и может быть улучшена в дальнейшем.

4 АПРОБАЦИЯ РЕЗУЛЬТАТОВ ИССЛЕДОВАНИЯ: ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ МОДЕЛИ

Глава посвящена экспериментальному исследованию разработанной модели. Рассматриваются обучение модели (параграф 4.1), а также оценка качества перевода по метрикам BLEU, METEOR, TER и анализ ошибок (параграф 4.2). В параграфе 4.3 приведены выводы по главе.

4.1 Обучение модели

Обучение модели проводилось на датасете OPUS-100, содержащем 1 миллион параллельных предложений. В связи с ограниченным объемом видеопамяти (GPU T4 с 16 ГБ) применялся механизм накопления градиентов с шагом 8, что позволило достичь эффективного размера батча, эквивалентного 32 предложениям при физическом размере батча 4. Обучение проводилось в течение 1 эпохи, что составило около 4 часов вычислительного времени.

Для оптимизации использовался алгоритм Adam с начальной скоростью обучения $\eta = 5 \times 10^{-5}$. В процессе обучения выполнялась валидация на отложенной выборке из 2000 предложений.

Значение валидационного loss составило 3.2843.

4.2 Оценка качества перевода: метрики BLEU, METEOR, TER, анализ ошибок

Для оценки качества перевода обученной модели были использованы три стандартные метрики: BLEU (Bilingual Evaluation Understudy) [6], METEOR (Metric for Evaluation of Translation with Explicit Ordering) [5] и TER (Translation Edit Rate) [1].

Оценка проводилась на тестовой выборке из 2000 предложений. Результаты представлены в таблице 4.1.

Таблица 4.1

Результаты оценки качества перевода

Метрика	Значение
BLEU	20.01
METEOR	0.3927
TER	1.0747

Для выявления типичных ошибок модели был проведен качественный анализ переводов. В таблице 4.2 приведены примеры переводов с указанием типа ошибки.

Анализ представленных примеров позволяет выделить следующие категории ошибок:

- А. **Семантические ошибки:** модель часто неправильно выбирает значение слов в контексте. Например, в первом примере фраза “вы просто не представляете” переводится как “you just don’t have to do it” вместо “you’ve no idea”; в третьем примере “обрушится” ошибочно переводится как “joke” вместо “collapse”. Особенно заметны ошибки при работе с датами и числами.
- В. **Повтор слов:** модель склонна к повторению слов и фраз, особенно в длинных предложениях. Примеры: “enough enough”, “school school”, “the amount of the amount”, “the home of the home”, “the Americans that had

Таблица 4.2

Анализ ошибок перевода

Исходное предложение (RU)	Эталонный перевод (EN)	Перевод модели (EN)	Тип ошибки
Было так холодно на улице, вы просто не представляете.	It's so cold out there, you've no idea.	It was so cold, you just don't have to do it.	Семантическая ошибка
Я всё знаю, мистер Уолкер.	Nothing gets by me, Mr. Walker.	I know everything, Mr. Walker.	Семантическая ошибка
Есть ли шанс, что все это просто обрушится?	Isn't there a chance this lot's just gonna collapse?	Is there anything that's just a joke?	Семантическая ошибка
Но два ученика — это не достаточно, чтобы держать открытой школу.	Two students are not enough to sustain a school.	But two — is not enough enough to keep the school school.	Повтор слов, грамматическая ошибка
Они знали о деньгах, и они знали точную сумму.	They knew about the money and they knew the exact amount.	They knew about money, and they knew the amount of the amount.	Повтор слов, тавтология
It is currently Fri May 20, 2016 12:29 am	It is currently Tue May 17, 2016 12:07 am	It is currently Fri May 27, 2016 1:16 am	Семантическая ошибка (даты)
Казалось, будто с ними легче разговаривать, чем с местными жителями, американцами, которые обслуживали нас и подавали нам еду.	It seemed they were probably easier to talk to than the local Americans who were waiting on us as and serving food.	It was like it was easier to talk to them than the Americans that had the Americans that had taken us and had brought us.	Повтор слов, потеря смысла
— Мы будем делать домашние задания.	— We'll do homework.	— We're gonna do the home of the home.	Семантическая ошибка, повтор слов

the Americans”. Это указывает на проблему с механизмом генерации, который не штрафует повторяющиеся токены.

С. Артефакты токенизации: ВРЕ-токенизатор разбивает слова на суб-токены, что приводит к появлению пробелов внутри слов в исходных текстах. В примерах изначально было: “представля ете”, “Уол кер”, “обру шится”, “Каза лось”, “жи телями”, “американ цами”, “обслужи вали”, “домаш ние зада ния”. Эта проблема связана с используемым претокенизатором Whitespace и отсутствием специальных маркеров для склеивания субтокенов при декодировании.

4.3 Выводы

Модель обучена на датасете OPUS-100. Валидационный loss составил 3.2843. Метрики: BLEU — 20.01, METEOR — 0.3927, TER — 1.0747.

Основные типы ошибок: семантические ошибки, повторы слов, артефакты токенизации.

Результаты подтверждают работоспособность модели и указывают на необходимость улучшения токенизатора, введения штрафа за повторения и увеличения данных для обучения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения летней научно-исследовательской практики была реализована и экспериментально исследована архитектура Transformer для задачи русско-английского машинного перевода.

Изучена архитектура Transformer, включая механизмы самовнимания, многоголовое внимание, остаточные связи и позиционное кодирование. Выполнена подготовка датасета OPUS-100: проведена фильтрация данных, обучен BPE-токенизатор с размером словаря 32 000 токенов, реализован пайплайн предобработки. Реализована архитектура Transformer с нуля на фреймворке PyTorch с размерностью $d_{model} = 512$, 6 слоями энкодера и декодера. Проведено обучение модели на 1 миллионе параллельных предложений. Валидационный loss составил 3.2843. Оценка качества перевода показала: BLEU — 20.01, METEOR — 0.3927, TER — 1.0747. Анализ ошибок выявил семантические ошибки, повторение слов и артефакты токенизации.

В рамках практики был предложен ряд рекомендаций по улучшению модели, реализация которых возможна при наличии более мощных вычислительных ресурсов (к примеру, Google Colab Pro). Ключевые направления улучшения включают замену токенизатора на ByteLevelBPETokenizer, введение механизма штрафа за повторения, увеличение размерности модели и расширение обучающей выборки.

Полученные результаты подтверждают работоспособность модели и могут быть применены при проектировании диалоговых ассистентов для перевода текстов. Автор выражает благодарность научному руководителю Антоняну Давиду Мелитосовичу за ценные консультации и помощь в выполнении работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. A Study of Translation Edit Rate with Targeted Human Annotation / M. Snover [и др.] // Proceedings of the 7th Conference of the Association for Machine Translation in the Americas (AMTA). — 2006. — С. 223—231. — URL: <https://aclanthology.org/2006.amta-papers.16/>.
2. *Alammar J.* The Illustrated Transformer. — 2018. — URL: <https://jalammar.github.io/illustrated-transformer/> (дата обращения: 28.06.2026).
3. *Archeon Knowledge Base.* Что такое токен и методы токенизации подслов. — 2024. — URL: https://kb.archeon.io/ai/what_token/ (дата обращения: 28.06.2026).
4. Attention is All You Need / A. Vaswani [и др.] // Advances in Neural Information Processing Systems. Т. 30. — Curran Associates, Inc., 2017. — С. 5998—6008. — URL: <http://arxiv.org/abs/1706.03762>.
5. *Banerjee S., Lavie A.* METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments // Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization. — 2005. — С. 65—72. — URL: <https://aclanthology.org/W05-0909/>.
6. BLEU: a Method for Automatic Evaluation of Machine Translation / K. Papineni [и др.] // Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL). — 2002. — С. 311—318. — URL: <https://aclanthology.org/P02-1040/>.
7. *Hugging Face.* Hugging Face Course: Tokenizers. — 2023. — URL: <https://github.com/huggingface/course/blob/main/chapters/ru/chapter6/8.mdx> (дата обращения: 28.06.2026).
8. *Lingvanex.* Нейронный машинный перевод: что это? — 2023. — URL: <https://lingvanex.com/ru/blog/what-is-neural-machine-translation/> (дата обращения: 28.06.2026).
9. *Sennrich R., Haddow B., Birch A.* Neural Machine Translation of Rare Words with Subword Units // Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL). — 2016. — С. 1715—1725. — URL: <https://arxiv.org/abs/1508.07909>.
10. *Technolady.* История машинного перевода: от Декарта до нейросетей. — 2021. — URL: <https://www.securitylab.ru/blog/personal/Technolady/355284.php> (дата обращения: 28.06.2026).

11. *Бюро переводов Б2Б*. История машинного перевода: развитие и появление. — 2023. — URL: <https://b2bperevod.ru/articles/tekhnologii-perevoda/istoriya-mashinnogo-perevoda/> (дата обращения: 28.06.2026).

12. *Википедия*. Нейронный машинный перевод. — 2026. — URL: https://ru.wikipedia.org/wiki/%D0%9D%D0%B5%D0%B9%D1%80%D0%BE%D0%BD%D0%BD%D1%8B%D0%B9_%D0%BC%D0%B0%D1%88%D0%B8%D0%BD%D0%BD%D1%8B%D0%B9_%D0%BF%D0%B5%D1%80%D0%B5%D0%B2%D0%BE%D0%B4 (дата обращения: 28.06.2026).

13. *Зотов А.* Машинный перевод. Как развивалась технология. — 2024. — URL: <https://habr.com/ru/articles/1003076/> (дата обращения: 28.06.2026).

14. *Иванов И. И., Петров П. П.* Обзор методов глубокого обучения для нейронного машинного перевода // Научно-технический вестник. — 2020. — URL: <http://www.nauteh-journal.ru/files/e986ec4f-45ec-4d6d-9dff-6541c7a0449f> (дата обращения: 28.06.2026).

15. *Кафедра лингвистики МГУ*. Системы машинного перевода. — 2019. — URL: <http://lingva.ffl.msu.ru/2019/08/%D1%81%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D1%8B-%D0%BC%D0%B0%D1%88%D0%B8%D0%BD%D0%BD%D0%BE%D0%B3%D0%BE-%D0%BF%D0%B5%D1%80%D0%B5%D0%B2%D0%BE%D0%B4%D0%B0/> (дата обращения: 28.06.2026).

16. *Козлов В. А.* Проектирование сервиса машинного перевода с использованием современных NMT архитектур // Электронная библиотека БГУ. — 2022. — URL: <https://elib.bsu.by/bitstream/123456789/328340/1/395-399.pdf> (дата обращения: 28.06.2026).

17. *Радиолоцман*. Что такое нейронный машинный перевод (NMT): история и виды. — 2022. — URL: <https://www.radiolocman.com/press-rel/rel.html?di=429-evolyuciya-perevoda-ot-pervyh-mashin-k-neyronnym-setyam> (дата обращения: 28.06.2026).

18. *Семенов С. С.* Обзор архитектуры рекуррентного Трансформера в контексте задач NLP // Труды МФТИ. — 2021. — Т. 64, № 1. — URL: https://mipt.ru/upload/%D0%A2%D1%80%D1%83%D0%B4%D1%8B_%D0%BC%D1%84%D1%82%D0%B8/64/01.pdf (дата обращения: 28.06.2026).

Приложение 1

Листинг программного кода

Ниже представлен полный код, использованный в данной работе, разделённый на файлы по функциональному назначению: подготовка данных и токенизация, реализация архитектуры модели, обучение, инференс и оценка качества перевода.

П1.1 Загрузка датасета и обучение токенизатора

Загружается англо-русская пара датасета OPUS-100, после чего обучается BPE-токенизатор. Поскольку в датасете обнаружались артефакты в виде китайских иероглифов и символов индийских алфавитов, перед обучением токенизатора данные фильтруются так, чтобы остались только латиница и кириллица. Тексты нарезаются на батчи, на которых и обучается токенизатор.

Листинг П1.1

Загрузка датасета OPUS-100 и обучение BPE-токенизатора

```

!pip install datasets

from datasets import load_dataset
5
dataset = load_dataset("Helsinki-NLP/opus-100", "en-ru")

print(dataset)

10 print(dataset['train'][:2])

from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
15 from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
tokenizer.pre_tokenizer = Whitespace() # нарезаем по пробелам
20
trainer = BpeTrainer(
    special_tokens=["[PAD]", "[UNK]", "[BOS]", "[EOS]"], # [
        PAD] - это пустышки, использую для выравнивания

```



```

vocab_size=32000 # это базовый размер, который использовал
                  и авторы статьи "Attention is all you need" для англо-ф
                  ранцузского перевода
25 )
import re

allowed_chars = re.compile(r"^[A-Za-zА-Яа-яЁё0-9\s.,!?:;()
30 ' \- \"] *$")

def is_clean(text):
    return bool(allowed_chars.match(text))

def batch_iterator(batch_size=1000):
35     for i in range(0, len(dataset["train"]), batch_size):
        batch = dataset["train"][i : i + batch_size]
        text_batch = []
        for item in batch["translation"]:
            if is_clean(item["en"]) and is_clean(item["ru"]):
40                 text_batch.append(item["en"])
                 text_batch.append(item["ru"])
        yield text_batch

tokenizer.train_from_iterator(batch_iterator(), trainer)
45
# tokenizer.save("opus100_bpe_tokenizer.json")

```

П1.2 Подготовка числовых данных

Тексты преобразуются в последовательности идентификаторов токенов: к русским (входным) последовательностям добавляется токен [EOS], к английским (целевым) — токены [BOS] и [EOS]. Поскольку предложения внутри батча имеют разную длину и не могут быть напрямую объединены в один тензор, реализован класс DataCollator, который динамически дополняет короткие предложения токенами [PAD] до длины самого длинного предложения в конкретном батче. Токенизированный датасет оборачивается в DataLoader, отвечающий за перемешивание и нарезку данных на батчи. Из-за добавленных токенов [PAD] в модель дополнительно передаются маски выравнивания (padding masks).

Подготовка числовых данных, DataCollator и DataLoader

```

# ID специальных токенов из токенизатора
pad_id = tokenizer.token_to_id("[PAD]")
bos_id = tokenizer.token_to_id("[BOS]")
5 eos_id = tokenizer.token_to_id("[EOS]")

MAX_LENGTH = 200

def preprocess_function(examples):
10     input_ids = []
    labels = []

    for item in examples["translation"]:
        # Токенизируем тексты и обрезаем их
15         ru_encoded = tokenizer.encode(item["ru"]).ids[:
            MAX_LENGTH]
        en_encoded = tokenizer.encode(item["en"]).ids[:
            MAX_LENGTH]

        # Вход энкодера (русский)
        ru_sequence = ru_encoded + [eos_id] # текст + [EOS]
20         # Выход декодера (английский)
        en_sequence = [bos_id] + en_encoded + [eos_id] # [BOS]
            + текст + [EOS]

        input_ids.append(ru_sequence)
25         labels.append(en_sequence)

    # 'input_ids' пойдут в энкодер, а 'labels' в декодер в
    # качестве таргета
    return {"input_ids": input_ids, "labels": labels}

30 tokenized_dataset = dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=["translation"]
)
35 print(tokenized_dataset)

print(tokenized_dataset['train'][:2])

```

```

40 import torch

class DataCollator:
    def __init__(self, pad_id):
        self.pad_id = pad_id

45
    def __call__(self, batch):
        # Извлекаем списки ID из батча
        input_ids = [item['input_ids'] for item in batch]
        labels = [item['labels'] for item in batch]

50
        # Находим максимальные длины в этом конкретном батче
        max_input_len = max(len(x) for x in input_ids)
        max_label_len = max(len(x) for x in labels)

55
        # Дополняем
        padded_inputs = [x + [self.pad_id] * (max_input_len -
            len(x)) for x in input_ids]
        padded_labels = [x + [self.pad_id] * (max_label_len -
            len(x)) for x in labels]

        return {
60
            'input_ids': torch.tensor(padded_inputs, dtype=
                torch.long),
            'labels': torch.tensor(padded_labels, dtype=torch.
                long)
        }

65 collator = DataCollator(pad_id=pad_id)

from torch.utils.data import DataLoader

70 BATCH_SIZE = 4

train_dataloader = DataLoader(
    tokenized_dataset["train"],
    batch_size=BATCH_SIZE,
75
    shuffle=True,
    collate_fn=collator
)

# Для валидации перемешивание не требуется
80 val_dataloader = DataLoader(

```

```

        tokenized_dataset["validation"],
        batch_size=BATCH_SIZE,
        shuffle=False,
        collate_fn=collator
85 )

# проверка
first_batch = next(iter(train_dataloader))
print("Размерность входных данных (RU):", first_batch['
    input_ids'].shape)
90 print("Размерность целевых данных (EN):", first_batch['labels'
    ].shape)

# Маска указывает True там, где стоит [PAD], чтобы attention и
    гнорировал эти позиции
src_key_padding_mask = (first_batch['input_ids'] == pad_id)
tgt_key_padding_mask = (first_batch['labels'] == pad_id)

```

П1.3 Код модели Transformer

Реализована архитектура Transformer с нуля на PyTorch. Каузальная маска (Causal Mask) заполняет верхний треугольник матрицы значением минус бесконечность, чтобы механизм внимания обнулял веса для будущих токенов декодера. Позиционное кодирование (Positional Encoding) реализовано на синусах и косинусах согласно статье *Attention Is All You Need* и подсказывает модели порядок токенов в последовательности, поскольку Transformer обрабатывает все токены параллельно. В Multi-Head Attention вычисления для всех голов внимания выполняются одной большой проекцией, после чего тензор «нарезается» операциями view и transpose. После каждого блока внимания в слоях энкодера и декодера следуют Position-wise Feed-Forward Network и структура Residual + LayerNorm. Итоговая модель SimpleTransformer объединяет стек слоёв энкодера и декодера в единую архитектуру.

Листинг П1.3

Реализация архитектуры Transformer

```

def generate_square_subsequent_mask(sz, device):
    # Создаем матрицу размера [sz, sz] из единиц, берем верхни
        й треугольник (без главной диагонали)
    mask = (torch.triu(torch.ones((sz, sz), device=device)) ==
        1).transpose(0, 1)

```

```

5      # Превращаем в float mask: 0.0      можно смотреть, -inf
      # нельзя смотреть
      mask = mask.float().masked_fill(mask == 0, float('-inf')).
          masked_fill(mask == 1, float(0.0))
      return mask

import torch
10 import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=1000):
15         super().__init__()
         pe = torch.zeros(max_len, d_model)
         position = torch.arange(0, max_len, dtype=torch.float)
             .unsqueeze(1)
         # Формула из статьи:  $10000^{(2i/d\_model)}$ 
         div_term = torch.exp(torch.arange(0, d_model, 2).float
             () * (-math.log(10000.0) / d_model))

20         pe[:, 0::2] = torch.sin(position * div_term) # Четные
            # индексы
         pe[:, 1::2] = torch.cos(position * div_term) # Нечетны
            # е индексы
         self.register_buffer('pe', pe.unsqueeze(0))

25     def forward(self, x):
         # x: [batch_size, seq_len, d_model]
         return x + self.pe[:, :x.size(1)]

class MultiHeadAttention(nn.Module):
30     def __init__(self, d_model, nhead):
         super().__init__()
         self.nhead = nhead
         self.d_k = d_model // nhead

35         # Проекции для получения Q, K, V и финального объедине
            # ния O
         self.w_q = nn.Linear(d_model, d_model)
         self.w_k = nn.Linear(d_model, d_model)
         self.w_v = nn.Linear(d_model, d_model)
         self.w_o = nn.Linear(d_model, d_model)

40     def forward(self, q, k, v, mask=None, key_padding_mask=
        None):

```

```

batch_size = q.size(0)

# Линейный слой + разделение на головы
# [batch_size, seq_len, d_model] в [batch_size, nhead,
45     seq_len, d_k]
Q = self.w_q(q).view(batch_size, -1, self.nhead, self.
    d_k).transpose(1, 2)
K = self.w_k(k).view(batch_size, -1, self.nhead, self.
    d_k).transpose(1, 2)
V = self.w_v(v).view(batch_size, -1, self.nhead, self.
    d_k).transpose(1, 2)

# Scaled Dot-Product Attention (Формула из статьи)
# scores: [batch_size, nhead, seq_len_q, seq_len_k]
50     scores = torch.matmul(Q, K.transpose(-2, -1)) / math.
        sqrt(self.d_k)

# Применяем каузальную маску (для декодера)
55     if mask is not None:
        scores = scores + mask.unsqueeze(0).unsqueeze(1)

# Применяем padding маску (скрываем [PAD] токены)
if key_padding_mask is not None:
60     scores = scores.masked_fill(key_padding_mask.
        unsqueeze(1).unsqueeze(2), float('-inf'))

# Softmax + умножение на V
attn_weights = torch.softmax(scores, dim=-1)
context = torch.matmul(attn_weights, V)
65

# Склеиваем головы обратно (Concat)
context = context.transpose(1, 2).contiguous().view(
    batch_size, -1, self.nhead * self.d_k)
return self.w_o(context)

70 class FeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward):
        super().__init__()
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.relu = nn.ReLU()
75     self.linear2 = nn.Linear(dim_feedforward, d_model)

    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))

```

```

80 class EncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, nhead)
        self.ff = FeedForward(d_model, dim_feedforward)
85     self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, src, src_padding_mask):
        # Attention + Residual + Norm
90     attn_out = self.self_attn(src, src, src,
        key_padding_mask=src_padding_mask)
        x = self.norm1(src + attn_out)

        # FeedForward + Residual + Norm
95     return self.norm2(x + self.ff(x))

class DecoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward):
        super().__init__()
100     self.self_attn = MultiHeadAttention(d_model, nhead)
        self.cross_attn = MultiHeadAttention(d_model, nhead)
        self.ff = FeedForward(d_model, dim_feedforward)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
105     self.norm3 = nn.LayerNorm(d_model)

    def forward(self, tgt, memory, tgt_mask, tgt_padding_mask,
        memory_padding_mask):
        # Masked Self-Attention
        attn1 = self.self_attn(tgt, tgt, tgt, mask=tgt_mask,
        key_padding_mask=tgt_padding_mask)
110     x = self.norm1(tgt + attn1)

        # Encoder-Decoder Cross-Attention (Query берем из x, a
        Key u Value из памяти кодера)
        attn2 = self.cross_attn(x, memory, memory,
        key_padding_mask=memory_padding_mask)
        x = self.norm2(x + attn2)
115     # FeedForward
        return self.norm3(x + self.ff(x))

class SimpleTransformer(nn.Module):

```

```

120 def __init__(self, vocab_size, d_model=512, nhead=8,
    num_layers=6, dim_feedforward=2048, max_len=1000):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, d_model)
    self.pos_encoder = PositionalEncoding(d_model, max_len
        =max_len)

125     # Создаем цепочки слоев через nn.ModuleList
    self.encoder_layers = nn.ModuleList([EncoderLayer(
        d_model, nhead, dim_feedforward) for _ in range(
            num_layers)])
    self.decoder_layers = nn.ModuleList([DecoderLayer(
        d_model, nhead, dim_feedforward) for _ in range(
            num_layers)])

    self.out_linear = nn.Linear(d_model, vocab_size)

130 def forward(self, src, tgt, src_padding_mask,
    tgt_padding_mask, tgt_mask):
    # Проход через Энкодер
    memory = self.pos_encoder(self.embedding(src))
    for layer in self.encoder_layers:
135         memory = layer(memory, src_padding_mask)

    # Проход через Декодер
    x = self.pos_encoder(self.embedding(tgt))
    for layer in self.decoder_layers:
140         x = layer(x, memory, tgt_mask, tgt_padding_mask,
            src_padding_mask)

    return self.out_linear(x)

```

П1.4 Код обучения

Задаются гиперпараметры модели и обучения. Для экономии видеопамати используется приём Accumulation Steps: вместо обновления весов после каждого батча градиенты накапливаются на протяжении нескольких батчей, и только после этого выполняется шаг оптимизатора — это позволяет использовать больший эффективный размер батча при ограниченной видеопамати. Также реализованы функции валидации модели, сохранения чекпоинтов (с возможностью восстано-

ния обучения) и визуализации кривых обучения, после чего запускается основной цикл обучения по эпохам.

Листинг П1.4

Гиперпараметры и цикл обучения модели

```

vocab_size = tokenizer.get_vocab_size()

# Гиперпараметры
5 D_MODEL = 512          # Embedding dimension
  NHEAD = 8              # Number of attention heads
  NUM_LAYERS = 6          # Number of encoder/decoder layers
  DIM_FEEDFORWARD = 2048 # Dimension of the feed-forward network

10 # для экономии памяти
  BATCH_SIZE = 4          # вместо 32 (помещается в память)
  ACCUMULATION_STEPS = 8  # 4 * 8 = 32 ( batch size)
  LEARNING_RATE = 5e-5    # чуть меньше для стабильности
  EPOCHS = 3              # оставляем как было

15 model = SimpleTransformer(
    vocab_size=vocab_size,
    d_model=D_MODEL,
    nhead=NHEAD,
20   num_layers=NUM_LAYERS,
    dim_feedforward=DIM_FEEDFORWARD,
    max_len=1000
)

25 # Move the model to the GPU if available (you're using a T4
    GPU)
  device = torch.device("cuda" if torch.cuda.is_available() else
    "cpu")
  model = model.to(device)
  print(f"Model is running on: {device}")

30 import torch.optim as optim
  import gc
  import matplotlib.pyplot as plt
  import os
  from IPython.display import clear_output

35 # папка для чекпоинтов
  CHECKPOINT_DIR = "checkpoints"
  os.makedirs(CHECKPOINT_DIR, exist_ok=True)

```

```

40 criterion = nn.CrossEntropyLoss(ignore_index=pad_id)
   optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

   train_losses = []
   val_losses = []
45 best_val_loss = float('inf')

   def validate(model, val_dataloader, criterion, device):
       model.eval()
       total_loss = 0

50       with torch.no_grad():
           for batch in val_dataloader:
               inputs = batch['input_ids'].to(device)
               labels = batch['labels'].to(device)

55               tgt_input = labels[:, :-1]
               tgt_out = labels[:, 1:]

               src_padding_mask = (inputs == pad_id).to(device)
               tgt_padding_mask = (tgt_input == pad_id).to(device)
60               )
               tgt_mask = generate_square_subsequent_mask(
                   tgt_input.size(1), device)

               logits = model(inputs, tgt_input, src_padding_mask
                               , tgt_padding_mask, tgt_mask)
               loss = criterion(logits.view(-1, logits.size(-1)),
                               tgt_out.contiguous().view(-1))
65               total_loss += loss.item()

       model.train()
       return total_loss / len(val_dataloader)

70   def save_checkpoint(model, optimizer, epoch, batch_idx, loss,
       is_best=False):
       checkpoint = {
           'epoch': epoch,
           'batch': batch_idx,
75           'model_state_dict': model.state_dict(),
           'optimizer_state_dict': optimizer.state_dict(),
           'loss': loss,
           'best_val_loss': best_val_loss,
           'train_losses': train_losses,

```

```

80         'val_losses': val_losses,
    }

    torch.save(checkpoint, os.path.join(CHECKPOINT_DIR, "
        checkpoint_latest.pt"))

85     if is_best:
        torch.save(model.state_dict(), os.path.join(
            CHECKPOINT_DIR, "best_model.pt"))
        print(f"Лучшая модель сохранена! Val Loss: {loss:.4f}"
              )

def plot_losses(train_losses, val_losses):
90     clear_output(wait=True) # Очищает вывод ячейки

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # График Train Loss
95     axes[0].plot(range(1, len(train_losses) + 1), train_losses
        , 'b-', linewidth=2, label='Train Loss')
    axes[0].set_xlabel('Эпоха')
    axes[0].set_ylabel('Loss')
    axes[0].set_title('Train Loss')
    axes[0].legend()
100    axes[0].grid(True, alpha=0.3)

    # График Validation Loss
    axes[1].plot(range(1, len(val_losses) + 1), val_losses, 'r
        -', linewidth=2, label='Validation Loss')
    axes[1].set_xlabel('Эпоха')
105    axes[1].set_ylabel('Loss')
    axes[1].set_title('Validation Loss')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    # Добавляем общий заголовок с лучшим loss
110    if val_losses:
        best_val = min(val_losses)
        best_epoch = val_losses.index(best_val) + 1
        fig.suptitle(f'Обучение модели | Лучший Val Loss: {
            best_val:.4f} (эпоха {best_epoch})', fontsize=14)

115    plt.tight_layout()
    plt.show()

```

```

checkpoint_latest_path = os.path.join(CHECKPOINT_DIR, "
    checkpoint_latest.pt")
120 start_epoch = 0
    start_batch = 0

    if os.path.exists(checkpoint_latest_path):
        print("Найден чекпоинт! Восстанавливаю")
125         checkpoint = torch.load(checkpoint_latest_path)

        model.load_state_dict(checkpoint['model_state_dict'])
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'
            ''])

130         start_epoch = checkpoint['epoch']
        start_batch = checkpoint['batch'] + 1
        best_val_loss = checkpoint.get('best_val_loss', float('inf'
            ''))
        train_losses = checkpoint.get('train_losses', [])
        val_losses = checkpoint.get('val_losses', [])

135         print(f"Восстановлено: эпоха {start_epoch}, батч {
            checkpoint['batch']}")

        # Показываем историю обучения
        if train_losses:
140             print(f"История: {len(train_losses)} эпох, лучший Val
                Loss: {min(val_losses):.4f}")
            plot_losses(train_losses, val_losses)
        else:
            print("Начинаем обучение с нуля")

145 CHECKPOINT_INTERVAL = 2000

    model.train()
    for epoch in range(start_epoch, EPOCHS):
        total_loss = 0
150         optimizer.zero_grad()

        for batch_idx, batch in enumerate(train_dataloader):
            # Пропускаем уже пройденные батчи
            if epoch == start_epoch and batch_idx < start_batch:
155                 continue

            # Forward
            inputs = batch['input_ids'].to(device)

```

```

labels = batch['labels'].to(device)

tgt_input = labels[:, :-1]
tgt_out = labels[:, 1:]

src_padding_mask = (inputs == pad_id).to(device)
tgt_padding_mask = (tgt_input == pad_id).to(device)
tgt_mask = generate_square_subsequent_mask(tgt_input.
size(1), device)

logits = model(inputs, tgt_input, src_padding_mask,
tgt_padding_mask, tgt_mask)
loss = criterion(logits.view(-1, logits.size(-1)),
tgt_out.contiguous().view(-1))

loss = loss / ACCUMULATION_STEPS
loss.backward()

if (batch_idx + 1) % ACCUMULATION_STEPS == 0:
optimizer.step()
optimizer.zero_grad()

total_loss += loss.item() * ACCUMULATION_STEPS

# Очистка памяти
if batch_idx % 50 == 0:
torch.cuda.empty_cache()
gc.collect()

# Чекпоинт
if batch_idx % CHECKPOINT_INTERVAL == 0 and batch_idx
> 0:
real_loss = loss.item() * ACCUMULATION_STEPS
save_checkpoint(
model, optimizer, epoch, batch_idx, real_loss,
is_best=False
)
print(f"Чекпоинт сохранен: эпоха {epoch}, батч {
batch_idx}, Loss: {real_loss:.4f}")

# Логги
if batch_idx % 100 == 0:
real_loss = loss.item() * ACCUMULATION_STEPS
print(f"Эпоха [{epoch+1}/{EPOCHS}], Батч {
batch_idx}, Loss: {real_loss:.4f}")

```

```

200     # итог эпохи
    avg_train_loss = total_loss / len(train_dataloader)
    train_losses.append(avg_train_loss)

    # валидация
    print("\nВыполняется валидация")
205     avg_val_loss = validate(model, val_dataloader, criterion,
                             device)
    val_losses.append(avg_val_loss)

    # результаты
    print(f"Итог Эпохи {epoch+1}:")
210     print(f"    Train Loss: {avg_train_loss:.4f}")
    print(f"    Val Loss:    {avg_val_loss:.4f}")

    # сохраняем лучшую
215     if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        save_checkpoint(
            model, optimizer, epoch, batch_idx, avg_val_loss,
            is_best=True
220         )

    plot_losses(train_losses, val_losses)

    torch.save(model.state_dict(), os.path.join(CHECKPOINT_DIR
        , f"model_epoch_{epoch+1}.pt"))
225     print(f"Модель эпохи {epoch+1} сохранена как model_epoch_{
        epoch+1}.pt\n")

    plot_losses(train_losses, val_losses)

    print(f"\nОбучение завершено!")
230     print(f"    Лучший Val Loss: {best_val_loss:.4f}")

```

П1.5 Код инференса

Реализована загрузка обученной модели из чекпоинта и функция пошагового (авторегрессивного) перевода текста: входной текст токенизируется, кодируется энкодером, после чего декодер генерирует целевую последовательность токенов за

токеном до появления токена [EOS]. Также реализованы функции для перевода набора примеров и интерактивного перевода в режиме диалога.

Листинг П1.5

Загрузка модели и инференс (перевод текста)

```

import torch
import torch.nn.functional as F

5
def load_model(model_path="checkpoints/best_model.pt"):
    model = SimpleTransformer(
        vocab_size=vocab_size,
        d_model=D_MODEL,
10
        nhead=NHEAD,
        num_layers=NUM_LAYERS,
        dim_feedforward=DIM_FEEDFORWARD,
        max_len=1000
    ).to(device)

15
    model.load_state_dict(torch.load(model_path))
    model.eval()
    print(f"Модель загружена из {model_path}")
    return model

20
def translate(model, text, max_len=100):
    model.eval()

    # Токенизируем входной текст
25
    ru_encoded = tokenizer.encode(text).ids

    # Обрезаем, если слишком длинный
    if len(ru_encoded) > 500:
        ru_encoded = ru_encoded[:500]

30
    # Добавляем [EOS]
    src_tokens = ru_encoded + [eos_id]
    src_tensor = torch.tensor([src_tokens], dtype=torch.long).
        to(device)

35
    # Создаем маску для энкодера
    src_padding_mask = (src_tensor == pad_id).to(device)

    # Прогоняем через энкодер
    with torch.no_grad():

```

```

40     memory = model.pos_encoder(model.embedding(src_tensor)
        )
        for layer in model.encoder_layers:
            memory = layer(memory, src_padding_mask)

    # Генерация перевода (пошагово)
45     tgt_tokens = [bos_id] # начинаем с [BOS]

    for _ in range(max_len):
        tgt_tensor = torch.tensor([tgt_tokens], dtype=torch.
            long).to(device)
        tgt_mask = generate_square_subsequent_mask(len(
            tgt_tokens), device)
50     tgt_padding_mask = (tgt_tensor == pad_id).to(device)

        with torch.no_grad():
            x = model.pos_encoder(model.embedding(tgt_tensor))
            for layer in model.decoder_layers:
155             x = layer(x, memory, tgt_mask,
                tgt_padding_mask, src_padding_mask)

            logits = model.out_linear(x[:, -1, :])

            # Берем токен с максимальной вероятностью (greedy)
60     next_token = torch.argmax(logits, dim=-1).item()

            # Если сгенерирован [EOS]      останавливаемся
            if next_token == eos_id:
                break

65     tgt_tokens.append(next_token)

    # Декодируем токены в текст
    translated_text = tokenizer.decode(tgt_tokens[1:])
70     return translated_text

def translate_examples(model, examples):
    print("примеры перевода")
    print()

75     for i, (ru_text, en_expected) in enumerate(examples, 1):
        print(f"Пример {i}:")
        print(f"    RU: {ru_text}")
        print(f"    EN (ожидаемый): {en_expected}")
80

```



```

        translated = translate(model, ru_text)
        print(f"    EN (перевод):    {translated}")

def interactive_translate(model):
85     print()
        print("интерактивный перевод")
        print()
        print("Введите текст на русском (или 'exit' для выхода):\n
            ")

90     while True:
        text = input("RU: ").strip()
        if text.lower() in ['exit', 'quit', 'выход']:
            print("До свидания!")
            break

95         if not text:
            continue

        translated = translate(model, text)
100        print(f"EN: {translated}\n")

# Загружаем лучшую модель
model = load_model("checkpoints/best_model.pt")

105 # Примеры для теста
test_examples = [
    ("Привет, как дела?", "Hello, how are you?"),
    ("Я люблю машинное обучение", "I love machine learning"),
    ("Сегодня хорошая погода", "Today is good weather"),
110    ("Это очень интересная задача", "This is a very
        interesting task"),
    ("Модель Transformer работает хорошо", "The Transformer
        model works well"),
]

# Переводим примеры
115 translate_examples(model, test_examples)

# Интерактивный режим (раскомментируйте, если хотите)
# interactive_translate(model)

```

П1.6 Код оценки модели

Качество перевода оценивается по метрикам BLEU, METEOR и TER на валидационной выборке. Метрика TER реализована через расстояние Левенштейна на уровне слов. Дополнительно реализована функция анализа ошибок, которая сопоставляет исходный текст, эталонный перевод и перевод модели на нескольких примерах.

Листинг П1.6

Расчёт метрик качества перевода (BLEU, METEOR, TER) и анализ ошибок

```

!pip install sacrebleu torchmetrics

import nltk
5 nltk.download('wordnet')
  nltk.download('omw-1.4')

!pip install sacrebleu

10 import sacrebleu
   from nltk.translate.meteor_score import meteor_score

def calculate_ter(hypotheses, references):
    total_edits = 0
15    total_words = 0

    for hyp, ref in zip(hypotheses, references):
        hyp_words = hyp.split()
        ref_words = ref.split()

20
        # Простейшая реализация TER через расстояние Левенштейна на уровне слов
        n = len(hyp_words)
        m = len(ref_words)

25
        dp = [[0] * (m + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            dp[i][0] = i
        for j in range(m + 1):
            dp[0][j] = j

30
        for i in range(1, n + 1):
            for j in range(1, m + 1):
                if hyp_words[i-1] == ref_words[j-1]:

```



```

logits = model.out_linear(x[:, -1, :])
next_token = torch.argmax(logits, dim=-1).item
    ()

    if next_token == eos_id:
        break
    tgt_tokens.append(next_token)

ref_text = tokenizer.decode(tgt[0].tolist())
ref_text = ref_text.replace('[BOS]', '').replace(
    [EOS]', '').replace('[PAD]', '').strip()

hyp_text = tokenizer.decode(tgt_tokens[1:])

references.append(ref_text)
hypotheses.append(hyp_text)

bleu = sacrebleu.corpus_bleu(hypotheses, [[ref] for ref in
    references])

meteor_scores = []
for hyp, ref in zip(hypotheses, references):
    score = meteor_score([ref.split()], hyp.split())
    meteor_scores.append(score)
meteor = sum(meteor_scores) / len(meteor_scores)

ter = calculate_ter(hypotheses, references)

print(f"BLEU: {bleu.score:.2f}")
print(f"METEOR: {meteor:.4f}")
print(f"TER: {ter:.4f}")

return bleu.score, meteor, ter

metrics = calculate_metrics(model, val_dataloader, max_samples
    =100)

import pandas as pd

results = {
    'Метрика': ['BLEU', 'METEOR', 'TER'],
    'Значение': [metrics[0], metrics[1], metrics[2]]
}
df = pd.DataFrame(results)

```

```

print(df.to_string(index=False))

115 def analyze_errors(model, dataloader, num_examples=10):
    model.eval()

    errors = []

120    with torch.no_grad():
        for batch_idx, batch in enumerate(dataloader):
            if batch_idx >= num_examples:
                break

125        src = batch['input_ids'][0:1].to(device)
        tgt = batch['labels'][0:1].to(device)

        src_text = tokenizer.decode(src[0].tolist())
        src_text = src_text.replace('[BOS]', '').replace('
        [EOS]', '').replace('[PAD]', '').strip()

130        ref_text = tokenizer.decode(tgt[0].tolist())
        ref_text = ref_text.replace('[BOS]', '').replace('
        [EOS]', '').replace('[PAD]', '').strip()

        memory = model.pos_encoder(model.embedding(src))
135        for layer in model.encoder_layers:
            memory = layer(memory, (src == pad_id).to(
                device))

        tgt_tokens = [bos_id]
        for _ in range(100):
140            tgt_tensor = torch.tensor([tgt_tokens], dtype=
                torch.long).to(device)
            tgt_mask = generate_square_subsequent_mask(len
                (tgt_tokens), device)
            tgt_padding_mask = (tgt_tensor == pad_id).to(
                device)
            src_padding_mask = (src == pad_id).to(device)

145            x = model.pos_encoder(model.embedding(
                tgt_tensor))
            for layer in model.decoder_layers:
                x = layer(x, memory, tgt_mask,
                    tgt_padding_mask, src_padding_mask)

            logits = model.out_linear(x[:, -1, :])

```

```

150         next_token = torch.argmax(logits, dim=-1).item
            ()

            if next_token == eos_id:
                break
            tgt_tokens.append(next_token)

155     hyp_text = tokenizer.decode(tgt_tokens[1:])

    errors.append({
        'src': src_text,
160         'ref': ref_text,
        'hyp': hyp_text
    })

    for i, error in enumerate(errors, 1):
165         print(f"\nПример {i}:")
        print(f"    Исходник: {error['src']}")
        print(f"    Референс: {error['ref']}")
        print(f"    Перевод: {error['hyp']}")

170     return errors

analyze_errors(model, val_dataloader, num_examples=10)

```